

HASHDIT

HashDit

Smart Contract Security

Venus Centrifuge Yield Group

Security Audit Report

Venus

31 Aug 2026 – 4 Sep 2026

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Period	31 Aug 2026 - 4 Sep 2026
Overall Risk	High
Project Name	Venus
Github	https://github.com/VenusProtocol/venus-liquidity-hub/pull/21
Commit	3450b9dc41dbdef1655965f21395bab20d982565

No.	Contract Name	Verdict	Details
1	AdapterCentrifuge.sol	High	[H01]-Fixed [L02]-Ack [L03]-Ack [I03]-Fixed
2	PriceGuards.sol	Medium	[M01]-Fixed [M02]-Fixed [L05]-Fixed [I01]-Ack [I04]-Ack
3	YieldGroupCentrifuge.sol	Low	[L01]-Fixed [L04]-Fixed [I01]-Ack [I06]-Fixed
4	tests/foundry/fork/centrifuge/	Informational	[I02]-Fixed
5	08_DeployCentrifuge.s.sol	Informational	[I05]-Fixed
6	06_DeployAssetVault.s.sol	Informational	[I05]-Fixed
7	GrantGuardianWildcard.s.sol	Low	[L06]-Fixed

Finding

[H01] Share and face-value counters are unbounded, allowing NAV inflation that drains the entire Hub	
Contract	AdapterCentrifuge.sol
Severity	**High**

Description

A Centrifuge position is valued as value = face + shares x price. All four price defences constrain price only; shares (the share quantity) and face (the asset-denominated counters) are never validated. Both are written by Centrifuge cross-chain settlement messages and are independent of the price.

```javascript
value = face;                                                    // face
counters: unvalidated
if (shares > 0) {
    uint256 observed = _pricePerShareOrZero(resource);
    uint256 price =
IYieldGroupCentrifuge(holder).guardedPrice(resource, observed);  //
price IS guarded
    if (price == 0) revert ZeroSharePrice(resource);
    value += (shares * price) / (10 **
IERC20Metadata(shareToken).decimals());       // shares:
unvalidated
}
```

The settlement message carries the two sides of the exchange as two adjacent, independent uint128 fields, and the BSC side books them without cross-checking. A single malicious or faulty fill can therefore issue an arbitrary share quantity against a small asset amount while the published price stays entirely honest, so none of the four defences reacts.

```javascript
// CB_FULFILLED_DEPOSIT payload (verified against the fork harness)
abi.encodePacked(
    CB_FULFILLED_DEPOSIT,
    _investor(),
    uint128(filledAssets),    // assets consumed
    uint128(issuedShares),    // shares issued  <- no relation
enforced to filledAssets
    uint128(cancelledAssets)
);
```

Impact

The Hub share price inflates with the overstated NAV. Any address holding Hub shares can redeem at the inflated price and withdraw the Hub's entire real liquidity, including assets in the Core, Flux and FRV yield groups that have nothing to do with Centrifuge. A negligible share balance is sufficient to trigger it. Second-order: _accrueFees mints excess performance fees against the inflated NAV and ratchets the high-water mark permanently, which does not revert once the manipulation is corrected.

Confirmed exploitable on a BSC mainnet fork at commit 3450b9d (share path: alice deposits 1000 and withdraws 1900, bob's withdrawable balance goes to

zero; face path: a 100 USDT cancel request inflated to 100 million). This requires counterparty-side failure (pool manager, cross-chain bridge or Centrifuge governance), which is the threat model the codebase itself states: "a pool manager can write any value".

Status note
This finding is known to the team and has been deliberately deferred; it is reported here because the deferral postpones the fix, not the exposure.

A PR reviewer independently described the same scenario and the same impact, concluding "Not saying we should do anything in this pr... in next regression we might wanna support resource / yieldGroup level net value bound". That places the mitigation in a later release, while the position it protects goes live in this one. Between those two points the Hub's entire liquidity — including the Core, Flux and FRV groups — depends on the Centrifuge settlement pipeline behaving correctly, with no on-chain check that it has.

Deferral is a scheduling decision, not a risk-acceptance one. The latter would name a sign-off owner, state the mitigating assumptions relied on at launch, set an initial exposure cap, and define what happens if those assumptions fail. None of the four is recorded. The remediation section below sets out why the fix is structurally awkward and what can be done in the meantime.

Recommendation

Add a value ceiling anchored on Venus's own net contribution rather than on price, and fail closed (revert) when it is exceeded. Do not clamp: clamping leaks value in the same way as the drop-guard snapshot issue and contradicts the Hub's established fail-closed principle.

Only three touch points are needed. Of the thirteen position state transitions, eleven are value-continuous (same-unit moves, or moves Venus itself initiates); only the two Centrifuge fill transitions are discontinuous, and both surface in totalAssets(), so a single check at the valuation exit covers both. Per-bucket monitoring is unnecessary.

JavaScript

```
// 1. on deposit routed   -> netContributed += amount
// 2. on withdraw delivered -> netContributed -= received
// 3. in totalAssets()    -> if (value > _maxAllowedValue(bound))
revert ValueBoundBreach(...);

// State can reuse the PriceGuards shape (per-resource snapshot +
rate + periodic
// re-anchor), swapping the anchor from "price" to "net contributed
assets".
struct ValueBound {
    uint128 netContributed;
    uint128 maxSnapshotValue;
    uint64  timestamp;
    uint32  interval;
    uint16  annualGrowthBps;
    uint16  gapBps;
}
```

Two implementation constraints explain the deferral and should be planned around. First, the bookkeeping must live in YieldGroupBase: its four value-moving entry points deposit (:260), withdraw (:283), depositResource (:320) and withdrawResource (:357) are not declared virtual, so YieldGroupCentrifuge cannot wrap them. This conflicts with the decision recorded in commit c36b645 ("Restore YieldGroupBase byte-identical to its pre-Centrifuge state"). We recommend bundling this change with the structural fix for the forceRemoveResource cleanup omission, since both touch the base and both require re-verifying storage layout and the full suite.

Second, and the real bottleneck: sizing the allowed growth requires data Venus does not hold. A value bound must accommodate the legitimate divergence between the fill price and the current mark, because Centrifuge fills at its own NAV snapshot while Venus marks at the current pricePerShare. Too tight and a routine fill triggers a false positive, which - since this reverts rather than clamps - halts the entire Hub; too loose and it does not stop the attack. The normal range of that divergence should be obtained from Centrifuge as historical fill data before the bound is sized.

	Interim measures, in order of strength: (1) do not launch with real funds until the bound is in place; (2) if launching first, sign a written risk acceptance covering the four items listed in the status note; (3) as a zero-sizing-risk stopgap, value the maxMint bucket at depositPrice instead of pricePerShare. Because depositPrice is the weighted average of the fills behind maxMint, maxMint x depositPrice equals the assets actually contributed, so quantity inflation cancels out on that bucket. This is partial only: it covers the settled-but-unclaimed window, and both fields come from the same message so a deliberate attacker can falsify both. Its value is raising the bar, since depositPrice can be cross-checked against pricePerShare by off-chain monitoring.

Status	Fixed in commit 85ab358 (refined by 78d16e7 and 2e3cf77). The price-based guard was replaced with a per-resource band on the position's own **value**, so an inflated share count is bounded exactly as an inflated price is. Venus's own flows are recorded by measured balance delta so they do not trip the band:
	
	JavaScript
	`// YieldGroupCentrifuge.totalAssets`
	`total += _guardedNav(resource, IResourceAdapter(adapter).totalAssets(resource, address(this)));`
	`// _dispatchDeposit — record what left, not what we asked`
	`_recordInflow(resource, balBefore > balAfter ? balBefore - balAfter : 0);`
	
	One deployment note. The band is opt-in per resource, as the contract header states, and nothing on-chain requires it: addResource does not check for a band and the deposit path does not either. Re-running the same fill with no band armed reports the full **10,100**, so a resource onboarded without setNavGuardRate carries H01 exactly as before the fix, silently — nothing reverts and no view distinguishes an unarmed resource from an empty one. deploy/bsctestnetwiring/ ships a complete, simulatable VIP payload for Core, Flux and FRV but not for Centrifuge, which is the one family whose onboarding carries this extra step. We suggest adding a WireCentrifugeResource.s.sol covering addResource, both inner queues, addYieldGroup, setNavGuardRate and the ACM role grants as a single payload.

[M01] The drop floor re-anchors on the unclamped price, so one over-cap print can raise it above the growth cap and halt the entire Hub	
Contract	PriceGuards.sol
Severity	Medium

Description

_updatePriceGuards re-anchors the two guards asymmetrically: the growth branch clamps the observed price to the cap first, the drop branch uses the raw price.

```javascript
  // PriceGuards.sol:333-335
  uint256 maxAllowed = _maxAllowedPrice(growthSnap);
  uint256 base = price > maxAllowed ? maxAllowed : price;   //
growth: clamped first
  uint256 next = base + (base * growthSnap.gapBps) / BPS;
  // PriceGuards.sol:346
  uint256 next = price - (price * dropSnap.gapBps) / BPS;   //
drop: raw price  <- defect
  dropSnap.minSnapshotPrice = uint128(next);
```

An over-cap print is correctly suppressed by the cap, so NAV is unaffected and no breach fires. The same print raises the floor using the value the cap has just rejected, and nothing bounds how far the floor may move in one interval. With an honest price of 1e18, growthGapBps = 100, dropGapBps = 200 and annualDropBps = 0, a one-day print of 2e18 gives:

```javascript
  new cap   ~= 1.020659e18   // from the clamped base
  new floor  = 1.960000e18   // from the raw 2e18 print
```

The floor now exceeds the cap, so no price satisfies both guards. Even where it does not, the corrected price sits below the poisoned floor and _guardedPrice reverts DropGuardBreach on every read. The breach branch at line 330 freezes both anchors while the breach holds, so later accruals cannot repair it.

Neither snapshot setter checks floor <= cap, so governance can write the same state directly.

Impact

Hub.totalAssets() has no try/catch, so the revert stops every Hub deposit, withdrawal, redemption and fee accrual, including for Core, Flux and FRV depositors with no Centrifuge exposure.

Recovery is not automatic: the floor decays only at annualDropBps (roughly 12.5 years here, never when annualDropBps == 0), so it takes two governance transactions — setDropGuardSnapshot to re-anchor, then unpauseHub. IPriceGuards.sol:15 claims a breach "lifts by itself once the price recovers", which is false once the floor is poisoned; in testing the price had fully recovered and the breach still held.

No attacker action is required or possible from the Venus side. The trigger is one inflated print — a bridge fault, a replayed cross-chain message, a pricing bug, or

	publisher-side manipulation. enforceDropGuard being permissionless, any address can additionally latch a Hub pause until governance repairs the state. No funds are lost; the failure is availability.

Recommendation	Anchor the floor on the same clamped base the growth guard already computes, so a price the cap rejected cannot become the floor's baseline:
	
	JavaScript
	`uint256 base = price;`
	`if (growthSnap.interval != 0) {`
	`uint256 maxAllowed = _maxAllowedPrice(growthSnap);`
	`if (base > maxAllowed) base = maxAllowed;`
	`}`
	`uint256 next = base - (base * dropSnap.gapBps) / BPS;  // drop branch`
	
	Also bound how far minSnapshotPrice may rise per interval, add a floor <= cap check to both snapshot setters, and correct IPriceGuards.sol:15 — a breach self-lifts only when the floor was not re-anchored on the bad print.

Status	Fixed in commit 6c11f57. The asymmetric re-anchor no longer exists: the replacement clamps the reading once and writes that single value to both stored numbers, so one side can no longer anchor on a value the other rejected:
	
	JavaScript
	`uint256 anchor = _clamp(observed, floor, cap);`
	`band.anchor = uint128(anchor);`
	`band.centre = uint128(anchor);`

[M02] A low print can reset the cap downward when only the Growth Guard is enabled	
Contract	PriceGuards.sol
Severity	Medium
Description	The Growth and Drop Guards may be enabled independently, but the low-price freeze only runs when the Drop Guard is enabled. In a Growth-only configuration, a single low print after the update interval becomes the Growth Guard's new base and lowers its cap. If the price then recovers, the recovered price is clamped to that depressed cap until the cap grows back or governance resets it. JavaScript `//contracts/YieldGroup/base/PriceGuards.sol:330`

```
if (dropOn && price < _minAllowedPrice(dropSnap)) return;

///contracts/YieldGroup/base/PriceGuards.sol:333-338

uint256 maxAllowed = _maxAllowedPrice(growthSnap);
uint256 base = price > maxAllowed ? maxAllowed : price;
uint256 next = base + (base * growthSnap.gapBps) / BPS;
growthSnap.maxSnapshotPrice = uint128(next);
```

Worked example: With an honest price of 1e18, an initial cap of 1.01e18, growthGapBps = 100(1%), and no Drop Guard, a 0.5e18 print produces:

```
None
base    = min(0.5e18, 1.01e18) = 0.5e18
new cap = 0.5e18 * 1.01         = 0.505e18

when price backed to 1e18, the according to the code

guardedPrice = min(1.00, 0.505) = 0.505(only 50% of original)
```

After the observed price returns to 1e18, valuation still returns 0.505e18.

Impact:
A transient low print can continue to understate NAV after the price recovers. In the example, the resource remains valued at 50.5% of its observed price, and the cap recovers only at its configured rate or through governance.

Recommendation	Give the Growth Guard its own downward-move check, or prevent automatic re-anchors from reducing its cap. Do not rely on the optional Drop Guard to protect Growth Guard state.
Status	Fixed in commit 6c11f57. The separate low-price freeze branch that could be inert is gone; a low reading is held against the floor before it can become the new centre, by the same _clamp shown above.

[L01] PriceExpired is set unilaterally by the pool manager and Venus cannot relax it	
Contract	YieldGroupCentrifuge.sol
Severity	Low
Description	Price freshness is checked in three layers. The middle one is Centrifuge's own expiry marker, enforced unconditionally.

JavaScript

```
  // (1) never published
  if (computedAt == 0) revert StalePrice(resource, leg, 0, maxAge);
  // (2) Centrifuge's own expiry - always enforced          <- this
finding
  if (block.timestamp > validUntil) revert PriceExpired(resource,
leg, validUntil);
  // (3) Venus's own cap - optional
  if (maxAge != 0 && computedAt < block.timestamp &&
block.timestamp - computedAt > maxAge) {
      revert StalePrice(resource, leg, computedAt, maxAge);
  }
```

Venus can only tighten, never relax. Check (2) runs before (3) and reverts unconditionally, so no value of Venus's own shareMaxAge / assetMaxAge can prevent it from firing. The only way around it is disablePriceAgeGuard, which switches off all three layers — including the one Venus does control.

validUntil is computedAt plus a maxAge set by the Centrifuge pool manager through Hub.setMaxAssetPriceAge — an _isManager-gated routine function, the same tier as setPoolMetadata. The motive is legitimate: Centrifuge enforces this expiry on its synchronous vault path (checkValidity = true) and skips it on the asynchronous one. Venus uses only asynchronous vaults.

Verified: maxAge applies retroactively
Confirmed against centrifuge/protocol-v3 at commit a1aeae93c94e8a3dbe078f0fefbe9a1a340ffde1 (the same commit the PR itself was source-verified against), src/core/spoke/Spoke.sol:

JavaScript

```
  // :313-317 - writes maxAge only, never touches computedAt
  function setMaxAssetPriceAge(PoolId poolId, ShareClassId scId,
AssetId assetId, uint64 maxPriceAge)
      external auth
  {
      _pricePoolPerAsset[poolId][scId][assetId].maxAge =
maxPriceAge;
  }
  // :289-305 - computedAt is updated only when a price is
published (a separate entry point)
```

```
  function updatePricePoolPerAsset(..., D18 price, uint64
computedAt) public auth {
      poolPerAsset.price = price;
      poolPerAsset.computedAt = computedAt;
  }
```

The two are independent entry points and validUntil() is derived as computedAt + maxAge. Setting maxAge therefore applies retroactively to the already-published price: it takes effect in the same block, with no assumption that anyone "forgot to republish". The same file at :298-300 explains today's state — on a first publication maxAge defaults to type(uint64).max, so markers never expire until that setter is called explicitly.

On-chain state (measured 2026-09-03, block timestamp 1788409057)
Spoke 0xEC3582fcDc34078a4B7a8c75a5a3AE46f48525aB. Both share legs read maxAge = uint64.max and are 1.68 days old; the asset legs are 286.57 days old (USDT) and 315.52 days old (USDC), also with maxAge = uint64.max. The setter has never been called for either fund.

The asset leg measures a stablecoin-to-pool-currency rate that barely moves, so Centrifuge republishes it roughly once a year. That leg is therefore permanently stale by construction and cannot satisfy any expiry window with a business meaning.

Impact
Two conditions are required, neither sufficient alone: Venus governance arms the price age guard (a routine hardening step at launch), and the Centrifuge pool manager sets a finite maxAge for a synchronous vault. The second is ordinary business for them, and they have no reason to know a Venus group reads the same markers.

Because maxAge applies retroactively and the asset leg is 286 days old, any value with a business meaning — hours to days — expires that leg in the block the configuration lands. YieldGroupCentrifuge.totalAssets() then reverts, Hub.totalAssets() reverts with it (no try/catch), and every Hub flow stops: deposits, withdrawals, redemptions and fee accrual. Core, Flux and FRV depositors are equally blocked, because the Hub cannot price itself.

Venus's only recourse is disablePriceAgeGuard, which gives up the whole guard including its own controllable layer. After recovery the resource has no price freshness protection at all until Centrifuge reverts the setting.

The trade is negative on every dimension: the protection gained is zero, since Centrifuge does not enforce this expiry on the asynchronous path Venus uses;

	adoption is zero, since all four measured legs sit at uint64.max; and what is retained is a full halt switch Venus cannot decline. Venus already covers the same threat — a price that is too old — with layer (3), which it fully controls. Note also that the loop in YieldGroupCentrifuge.totalAssets has no per-resource try/catch, so PriceExpired on a single resource reverts the valuation of the whole group, and with it the whole Hub.
Recommendation	Remove the check. Venus's own freshness cap covers the same threat and is fully under its control, whereas validUntil is set unilaterally by Centrifuge. If it is kept out of deference to the publisher, put it behind a Venus-controlled switch: JavaScript `if (guard.enforcePublisherWindow && block.timestamp > validUntil) { revert PriceExpired(resource, leg, validUntil); }` Correct the NatSpec in ICentrifugeSpoke.sol, which claims validUntil reads as type(uint64).max while maxAge is unset. Measurement shows all three fields read zero when a leg has never been published, which is handled by the computedAt == 0 branch instead. Before launch, ask Centrifuge whether JTRSY or JAAA are scheduled to get a synchronous vault. Independently, poll the maxAge returned by markersPricePoolPerAsset / markersPricePoolPerShare and alert when it changes from uint64.max to a finite value - readable on-chain at no cost, and the only signal available before the halt.
Status	Fixed in commit c8b595c. The price age guard was removed entirely, so Centrifuge's validUntil is no longer read and cannot halt Venus valuation.

[L02] baseManager is mutable, so a manager migration silently zeroes the valuation	
Contract	AdapterCentrifuge.sol
Severity	Low

Description

Every position read resolves the manager freshly from the vault and is never cached against a recorded value.

```JavaScript
function _investments(address resource, address holder)
    internal
    view
    returns (ICentrifugeAsyncRequestManager.AsyncInvestmentState
memory)
{
    address manager =
ICentrifugeAsyncVault(resource).baseManager();   // re-read every
call
    return
ICentrifugeAsyncRequestManager(manager).investments(resource,
holder);
}
```

baseManager is a writable storage variable on the vault (file(bytes32,address) confirmed on-chain, callable by Root). If Centrifuge repoints the manager without migrating the per-position investments[vault][group] entry, the new manager returns a fully-formed all-zero struct - the ordinary result of reading an unwritten mapping key - which decodes successfully and Venus accepts as truth. Venus cannot distinguish "the new manager has no record" from "the position is genuinely empty", because the returned bytes are identical; validation is therefore only possible against the manager address, not the returned data.

Impact

Six of the seven valuation buckets zero instantly, leaving only the held share balance - the one bucket that does not live in the manager's storage. NAV drops on the spot, depositors in that block mint against the depressed price and permanently dilute existing holders, while the vanished assets are still held in Centrifuge escrow. There is no revert, no event, and none of the four price defences reacts. receiptBalance zeroes in step, so removeResource would consider the position cleared. This requires Centrifuge to perform a repoint-without-state-migration, which would affect all of its own investors too and therefore carries a strong incentive against - hence Low likelihood.

Recommendation	Record the manager address at registration and verify it on every read, reverting rather than silently following the new pointer. JavaScript `if (manager != expectedManager[resource]) revert ManagerChanged(resource, manager);` The PR already applies this snapshot-and-verify pattern to poolId / scId / assetId in setPriceAgeGuard; the manager pointer is not treated equivalently. Note the fix is nearly free here because _investments already reads baseManager - adding an address comparison costs no extra external call. Operationally, claiming promptly is a partial mitigation: the held share balance survives a manager migration precisely because it is not manager-side state, so the less value sits in maxMint, the smaller the exposure.
Status	Acknowledged. Accepted as a counterparty dependency owned by Centrifuge governance and monitored off-chain. One correction to the stated reasoning: the new NAV band does not weaken this finding. _guardedNav returns early on a zero reading (if (observedNav == 0) return 0;), so a manager migration that zeroes all seven buckets bypasses the floor entirely and receives no mitigation at all.

[L03] Share-token uniqueness is not validated, allowing double counting	
Contract	AdapterCentrifuge.sol
Severity	Low

Description	The group's entire shareToken.balanceOf is attributed to every registered resource. The adapter resolves the share token from the vault on each valuation and adds the full balance:
	
	JavaScript
	`shareToken = ICentrifugeAsyncVault(resource).share();`
	`shares = IERC20(shareToken).balanceOf(holder) + uint256(inv.pendingRedeemRequest) + inv.maxMint + inv.claimableCancelRedeemRequest;`
	
	totalAssets() runs this once per registered resource, so two resources resolving to the same share token each claim the whole balance.
	
	Several Centrifuge vaults share one share token. A share class has exactly one share token but can be served by multiple vaults, and the vault registry keys on the request manager as well as the asset (centrifuge/protocol-v3 @ a1aeae93c94e8a3dbe078f0fefbe9a1a340ffde1,src/core/spoke/VaultRegistry.sol:27):

```
JavaScript
mapping(PoolId => mapping(ShareClassId => mapping(AssetId =>
mapping(IRequestManager => IVault))))
      public vault;
```

Because the manager is part of the key, more than one live vault can exist for the same (pool, share class, asset) triple, and linkVault only forbids re-linking the same vault rather than occupying a second slot. Confirmed on-chain: the JTRSY/USDT and JTRSY/USDC vaults report identical share(), poolId, scId and baseManager, differing only in the accepted asset.

Neither addResource nor validateRegistration checks whether a candidate vault's share() is already claimed by a registered resource. Note that this is not the same vault added twice but two distinct addresses, so the already-registered guard never fires. A sibling vault also passes the remaining gates: its asset() matches the group's, and validateRegistration only probes share(), baseManager(), investments() and pricePerShare(), all of which it satisfies.

Impact
The realistic trigger is a Centrifuge vault migration: a new request manager, and a new vault linked for the same fund and asset. The natural ordering, add the new resource before removing the old one, necessarily opens the window, and addResource succeeds silently with no warning.

While both are registered the same holding is counted twice and the group's NAV doubles, carrying a full duplicate onto Hub NAV. No price moves and no price defence reacts: this is a quantity defect and therefore outside the scope of all four price guards. Depositors entering during the window mint against the inflated NAV and permanently dilute existing LPs.

Recovery is also worse than it should be. Both resources report a non-zero receiptBalance, because both read the same underlying balance, so removeResource reverts for either one. The only exit is forceRemoveResource, which requires pausing the Hub, so a routine vault upgrade forces governance into a protocol-wide pause.

Recommendation	Reject a vault whose share() matches that of any already-registered resource. Because the base addResource is external, this needs a hook in YieldGroupBase such as _validateResourceAddition. Operationally, a vault migration must be completed within a single batch: pause, add the new vault, forceRemoveResource the old one, unpause. Splitting it across transactions leaves both registered and triggers this finding.
Status	Acknowledged. Accepted with operational controls (advance notice of a migration, single-transaction VIP, share-balance checks during simulation). One correction: the first argument given does not hold. Centrifuge's registry keys on

	[poolId][scId][assetId][requestManager], so two vaults can share the same underlying asset and differ only by manager — which is the migration case the finding describes. The conclusion stands; we suggest resting it on the three operational controls instead.

[L04] The price-age guard lacks the shares > 0 condition and can block a position that needs no price	
Contract	YieldGroupCentrifuge.sol
Severity	Low

Description	Three checks on the valuation path can revert, and their scopes are inconsistent. ZeroSharePrice (AdapterCentrifuge.sol:232) and DropGuardBreach (via guardedPrice, called from :229) both sit inside the shares > 0 branch. StalePrice / PriceExpired does not.
	
	JavaScript
	`_enforcePriceAgeGuard(resource, adapter);`
	`total += IResourceAdapter(adapter).totalAssets(resource, address(this));`
	
	Both legs the age guard validates feed only pricePerShare(), which is used only for the share-denominated buckets. The three face-value buckets (pendingDepositRequest, claimableCancelDepositRequest, maxWithdraw) are already in asset units and pass through no price at all. So when shares == 0 the guard blocks on a price the valuation never reads.
	The property asserted at AdapterCentrifuge.sol:220-223 is therefore false for the full valuation path: "a position holding nothing share-shaped is priced without reading the feed or the guards at all, and neither a dead feed nor a breached floor can take it down."
	
	Impact
	YieldGroupCentrifuge.totalAssets() reverts, so Hub.totalAssets() (no try/catch) reverts and the entire Hub halts, including Core / Flux / FRV depositors. One governance transaction (disablePriceAgeGuard) recovers it.
	Two reachable paths. First, an unsettled new position: the deposit sits in pendingDepositRequest, Centrifuge has not settled, and the feed is stale or the share class has not begun publishing. Such a position has never held a share and ZeroSharePrice never appears. Second, a fully wound-down position: the redemption has settled and the value sits in maxWithdraw awaiting a claim - and a wound-down share class is exactly what Centrifuge is most likely to stop publishing a price for, so the two conditions naturally coincide.
	The second path also defeats the documented escape from ZeroSharePrice a second time: even once Centrifuge settles the redemption and the position becomes all face-value, ZeroSharePrice stops firing but the same dead feed then

	blocks on StalePrice instead, requiring a second governance transaction that is not mentioned in any documentation. This only occurs when the age guard is armed (if (!guard.enabled) return;) - arming this optional defence is itself what creates the failure mode.

Recommendation	Move the age check inside the shares > 0 branch so all three checks share one scope. The adapter already establishes the callback-into-group pattern for guardedPrice, so the same shape applies; then remove the unconditional _enforcePriceAgeGuard call from totalAssets().
	
	JavaScript
	`if (shares > 0) {`
	`  IYieldGroupCentrifuge(holder).enforcePriceAge(resource);  // moved here`
	`  uint256 observed = _pricePerShareOrZero(resource);`
	`  uint256 price = IYieldGroupCentrifuge(holder).guardedPrice(resource, observed);`
	`  ...`
	`}`
	
	Minimum without a structural change: correct the NatSpec at AdapterCentrifuge.sol:220-223 and at YieldGroupCentrifuge.sol:305-309 (the latter lists three revert sources without noting that only StalePrice is independent of bucket composition), and bind disablePriceAgeGuard into the wind-down runbook.

Status	Fixed in commit c8b595c. With the price age guard removed, totalAssets() no longer performs an unconditional staleness check ahead of the shares > 0 branch.

[L05] The drop breaker fires on a price the position does not depend on, pausing a healthy Hub	
Contract	PriceGuards.sol
Severity	Low

Description	totalAssets() only reads the price inside the shares > 0 branch: a position holding no shares is valued entirely from its face-denominated buckets and never consults the feed. enforceDropGuard has no such condition — it compares the raw price against the floor and pauses the Hub on the result.
	
	JavaScript
	`// PriceGuards.sol:68-79`
	`uint256 price = _observedPrice(resource);`
	`if (price == 0) return false;`
	`uint256 minAllowed = _minAllowedPrice(dropSnap);`
	`if (price >= minAllowed) return false;  // no `shares > 0` condition`
	`emit DropGuardBreached(resource, price, minAllowed);`

```
  _onDropBreach(resource, price, minAllowed);    // ->
IHub(_hub).pauseHub()
```

So the two paths can disagree about the same resource. A position that has been fully redeemed holds only assets; the fund's price can fall arbitrarily without affecting it, and totalAssets() correctly reports its value. If the resource is still registered with its drop guard armed, monitoring polling enforceDropGuard across registered resources will fire the breaker on a price that no longer bears on anything Venus holds.

This is not an access-control issue: the function is permissionless by design, and the caller influences neither the price nor the floor. The defect is that the trigger condition does not distinguish a price that matters from one that does not.

Impact
The Hub pauses — deposits, withdrawals, redemptions and fee accrual for every family — while nothing is actually wrong. unpauseHub is VIP, so the outage lasts as long as governance takes.

Reachability is narrow: it needs a resource wound down to zero shares but still registered with an armed guard. That state persists because a normal removeResource leaves the guard configuration in storage — only forceRemoveResource clears it.

Recommendation	Require non-zero share exposure before firing the breaker, so it and valuation agree on when a price is meaningful. See also L04, the same scoping gap in the valuation path.
Status	Fixed in commit 6c11f57. The permissionless breaker was removed; the replacement clamps valuation into the band and never pauses the Hub:

JavaScript
```
  if (floorOn && observedNav < floor) return floor;
  if (capOn && observedNav > cap) return cap;
  return observedNav;
```

[L06] The GrantGuardianWildcard deployment script omits the Centrifuge family's eleven ACM roles	
Contract	GrantGuardianWildcard.s.sol
Severity	Low

Description

This script grants the testnet Guardian a batch of wildcard ACM roles. Its signature list is maintained in sections, one per contract, with a section for each YieldGroup family. The script's own NatSpec scopes the list to Hub.sol, YieldGroupBase.sol, YieldGroup.sol, YieldGroupFRV.sol and HubRegistry.sol — the enumeration stops at FRV, so the newly added YieldGroupCentrifuge.sol and NavGuard.sol are not covered and the corresponding section is absent.

```JavaScript
  // GrantGuardianWildcard — _roleStrings()
  sigs = new string[](32);
  ...
  // YieldGroup.sol (Core / Flux: caps + blocks per year).
  sigs[k++] = "raiseResourceCap(address,uint256)";
  sigs[k++] = "lowerResourceCap(address,uint256)";
  sigs[k++] = "setBlocksPerYear(uint256)";
  // YieldGroupFRV.sol (no caps; adds forceRemoveResource).
  sigs[k++] = "forceRemoveResource(address)";
  // ← the YieldGroupCentrifuge.sol section is missing here
  // HubRegistry.sol (...)
  sigs[k++] = "addHub(address)";
  sigs[k++] = "removeHub(address)";
  require(k == sigs.length, "GrantGuardianWildcard: role count
mismatch");
```

Diffing the ACM strings the family actually checks (YieldGroupCentrifuge._checkAccessAllowed and NavGuard._requireGuardConfig) against the thirty-two the script grants leaves eleven of the twelve Centrifuge strings uncovered:

```JavaScript
  setNavGuardRate(address,uint16,uint16,uint16,uint32,bool,bool)
  setNavGuardSnapshot(address,uint128,uint64)
  setNavGuardEnabled(address,bool,bool)
  requestRedeem(address,uint256)
  cancelDepositRequest(address)
  cancelRedeemRequest(address)
  claimDeposit(address)
  claimRedeem(address)
  claimCancelDeposit(address)
  claimCancelRedeem(address)
  setSpotAPYBps(address,uint64)
```

The one that is covered, forceRemoveResource(address), is covered by coincidence: the FRV section already supplies that string, and a wildcard role key does not distinguish the target contract. Because the wildcard matches on the signature string, the functions Centrifuge inherits from YieldGroupBase —

addResource, both inner queue setters, pauseResource and so on — work, while none of the eleven functions unique to this family do.

Impact:
The granted and the missing split cleanly along a functional boundary. The Guardian **can** register a Centrifuge vault, configure the deposit and withdraw queues, attach the group to the Hub, and begin routing user funds into it. The Guardian **cannot** arm the NAV band, open a redemption request, or claim any settled amount.

The resulting state is a resource that is funded, unprotected, and unexitable:

- **Unprotected** — with no band armed, _guardedNav returns the counterparty's figure verbatim. Verified: a settlement issuing 10,000 shares against 1 asset consumed is reported as 10,100 rather than 103, leaving the original H01 quantity inflation entirely unbounded.
- **Unexitable** — an asynchronous vault requires requestRedeem before anything lands in maxWithdraw, and neither that role nor the four claim* roles are granted, so routed funds stay put until a second grant batch lands.

README.md lists all eleven signatures correctly and states the hazard outright: an onboarding VIP that skips the three setNavGuard* roles leaves that resource's NAV band unarmed for good, and an unarmed band reports the counterparty's value to the Hub unclamped. The documentation is right; the script contradicts it.

Scope is limited to testnet: execute() refuses any chain other than 97, and mainnet role grants are performed by a VIP outside this repository. The script is nonetheless the only executable, simulate()-able role-grant artifact in the repo, and therefore the most likely template for that VIP.

Recommendation

Add the Centrifuge section after the FRV section and raise the array length from 32 to 43:

```JavaScript
  // YieldGroupCentrifuge.sol (async lifecycle + the NAV band).
  sigs[k++] =
"setNavGuardRate(address,uint16,uint16,uint16,uint32,bool,bool)";
  sigs[k++] = "setNavGuardSnapshot(address,uint128,uint64)";
  sigs[k++] = "setNavGuardEnabled(address,bool,bool)";
  sigs[k++] = "requestRedeem(address,uint256)";
  ... // and the remaining seven
```

Better still, remove the duplication: CentrifugeBase.t.sol's _allRoles() already maintains the correct list, and the two should share one constant so they cannot diverge again. Consider also adding a post-condition to simulate() asserting that every _checkAccessAllowed signature on the target YieldGroup resolves.

Status	Fixed in commit 411aa65. The script now grants all 11 missing Centrifuge signatures — the eight async request/claim/cancel entry points, setSpotAPYBps, and the three NavGuard setters — bumping the sigs array from 32 to 43.

[I01] Guard parameters have no upper or lower bounds

Contract	PriceGuards.sol (_setGrowthGuardRate / _setDropGuardRate), YieldGroupCentrifuge.sol
Severity	Informational

Description	The two rate setters validate asymmetrically: the drop guard bounds gapBps, the growth guard does not bound it at all. Neither bounds the annual rate, and interval is only checked against zero on both sides.
	
	JavaScript
	`// growth: gapBps entirely unbounded`
	`if (annualGrowthBps == 0 || interval == 0) revert InvalidGrowthGuardRate();`
	
	`// drop: gapBps IS bounded`
	`if (interval == 0 || gapBps >= BPS) revert InvalidDropGuardRate();`
	
	Impact
	interval = 1 combined with a large gapBps lets the cap compound every block and go inert within a single block; annualGrowthBps / annualDropBps are uint16 with no ceiling. Separately, the Centrifuge family bypasses _resourceCap entirely because it overrides _resourceDepositRoom, so there is no per-resource exposure cap and multiple vaults in one group cannot be limited individually.
	
	A further documentation mismatch: under a sustained over-cap price the effective annual bound exceeds the nominal annualGrowthBps, because gapBps is re-applied every interval rather than acting as the one-off buffer the NatSpec describes. The per-interval compounding factor is (1 + annualGrowthBps x interval/year) x (1 + gapBps). With the NatSpec's own recommended (annualGrowthBps=1000, gapBps=100, interval=30 days) the documented value is 986 bps while the measured bound is 2431 bps - a factor of 2.46, independently reproduced from the formula. The factor is not universal and grows sharply as interval shortens: the same 1% gap at interval = 1 day contributes roughly 37.8x on its own.

Recommendation	Bound gapBps and both annual rates, set a floor on interval, and add a joint check relating interval to gapBps. Add a per-resource exposure cap for this family. Correct the growth-cap documentation, or change the semantics so gapBps counts only when the price is the binding constraint.

Status	Acknowledged. Accepted on the grounds that the setters are ACM-gated and VIP-only. The finding carries over to the new guard unchanged: driftBps has no upper bound and nothing relates it to upGapBps and interval, while the gap re-applies at every re-anchor. Measured: 8 %/yr drift, a 2 % gap and daily re-anchoring permit **152,195 %** over one year. Governance gating addresses the missing bounds but not the divergence between the documented rate and the effective one — which is the half review cannot catch, because each individual value looks conservative.

[I02] The fork suite never arms the growth or drop guard	
Contract	tests/foundry/fork/centrifuge/
Severity	Informational

Description	Across the 21 tracked fork test files delivered with the PR, setGrowthGuardRate, setDropGuardRate, setGrowthGuardSnapshot, setDropGuardSnapshot and enforceDropGuard have zero occurrences. Unit-level coverage is thorough by contrast (DropGuard.t.sol 50 occurrences, GrowthGuard.t.sol 38).
	So the guard logic is well validated, but never against the real cross-chain price pipeline: unit tests drive price through MockCentrifugeVault.setPrice(), a plain setter, whereas on fork the price arrives through the Spoke marker pipeline.
	
	JavaScript
	`// The breaker path with zero fork coverage:`
	`function _onDropBreach(address, uint256, uint256) internal override`
	`{`
	`  IHub(_hub).pauseHub();`
	`}`
	
	`// Hub.pauseHub :386-387`
	`function pauseHub() external override {`
	`  _checkAccessAllowed("pauseHub()");  // never exercised from a real breach`
	`  ...`
	
	Impact
	The circuit breaker is the weakest-verified link. The full path enforceDropGuard -> _onDropBreach -> IHub.pauseHub() -> Hub._checkAccessAllowed("pauseHub()") has zero occurrences across the fork suite; unit tests go through MockHubStub instead. We exercised the path mechanically at unit level during this review and it does complete, so the guard arithmetic itself is sound. What remains unverified is the part that cannot be checked from this repository: the ACM role the breaker depends on is not present in the deploy configuration, so the defence has never been run end to end against a real Hub with a real permission configuration. Per IPriceGuards.sol:283-284, if that role is missing the breaker does not degrade to

	an alarm — the whole call reverts and the breach goes unrecorded, because the DropGuardBreached event rolls back with it.
Recommendation	Add fork coverage for both guards: an inflated print followed by a price recovery, deposits and withdrawals while the cap is suppressing, and recovery after a breach. Include the full breaker path against a real Hub with the ACM role granted, so the role dependency is verified rather than assumed.
Status	Fixed in commit 85ab358. Fork_CentrifugeGuards.t.sol arms the band against BSC mainnet across five scenarios, including test_aClampedPositionNeverPausesTheRealHub.

[I03] The documented escape route from ZeroSharePrice does not work	
Contract	AdapterCentrifuge.sol
Severity	Informational
Description	The error's NatSpec tells an operator how to clear a ZeroSharePrice halt.

JavaScript

```
/**
 * @notice Thrown when a position contains shares but the vault
reports a zero share price.
 * @dev `disablePriceAgeGuard` does not clear this: it is a
separate check from the staleness guard.
 *      To get out, redeem the shares - `requestRedeem` and the
claims never read `totalAssets`,
 *      so they still work, and the check stops firing once the
position is all in asset units.
 *      If Centrifuge never settles, `forceRemoveResource` writes
it off.
 */
```

```
error ZeroSharePrice(address resource);
```

requestRedeem only moves value from balanceOf into pendingRedeemRequest, and both sit in the same share-denominated subtotal, so shares > 0 still holds and the revert persists.

JavaScript

```
shares = IERC20(shareToken).balanceOf(holder) +
uint256(inv.pendingRedeemRequest) + inv.maxMint
    + inv.claimableCancelRedeemRequest;   // requestRedeem moves
value WITHIN this sum
```

The only in-protocol exits are Centrifuge settling the redemption - least likely precisely when the feed is dead - or forceRemoveResource.

	Impact The code behaviour (fail-closed) is correct, but the documentation promises a recovery path that does not exist, in a runbook for recovering from a protocol-wide halt. During an incident this wastes time and misdirects diagnosis. See also [L04]: even after reaching the all-asset-units state, an armed price-age guard continues to block with StalePrice, so escaping requires a second governance transaction (disablePriceAgeGuard).
Recommendation	Correct the comment: the only in-protocol exits are Centrifuge settlement or forceRemoveResource, and requestRedeem does not clear the check. If a genuine Venus-side self-recovery path is wanted, it requires either valuing the share buckets at the recorded depositPrice / redeemPrice when the feed fails, or a governance switch that marks the share buckets at zero - a product decision.

Status	Fixed in commit [85ab358]. The NatSpec now states that requestRedeem does not clear the condition and names the two real remedies:
	JavaScript
	`* @dev \`requestRedeem\` does not clear this: it still works, but a pending redeem is counted as`
	`* shares. This clears when Centrifuge settles that redeem and the value lands in`
	`* \`maxWithdraw\`... If Centrifuge never settles, \`forceRemoveResource\` writes the position off.`

[I04] setDropGuardSnapshot accepts a backdated timestamp, which can silently neutralise the drop guard	
Contract	PriceGuards.sol
Severity	Informational

Description	JavaScript
	`The snapshot setter rejects a zero and a future timestamp, but accepts any past value.`
	`if (dropSnap.interval == 0) revert InvalidDropGuardSnapshot();`
	`if (minSnapshotPrice == 0) revert InvalidDropGuardSnapshot();`
	`if (timestamp == 0 || timestamp > block.timestamp) revert InvalidDropGuardSnapshot();  // past is allowed`
	
	`dropSnap.minSnapshotPrice = minSnapshotPrice;`
	`dropSnap.timestamp = timestamp;`
	
	The live floor decays linearly from timestamp, so the further back the timestamp is set, the more decay has already accrued and the lower the effective floor.
	
	JavaScript
	`uint256 decay = (minSnapshotPrice * dropSnap.annualDropBps * elapsed) / (BPS * SECONDS_PER_YEAR);`
	`return decay >= minSnapshotPrice ? 0 : minSnapshotPrice - decay;`
	
	Impact
	Governance can set a nominally correct minSnapshotPrice while a backdated timestamp pushes the effective floor arbitrarily low - to zero if the backdating exceeds the full decay horizon - leaving a guard that reads as armed but can never fire. There is no event or view that surfaces the discrepancy between the configured anchor and the effective floor.

Recommendation	Bound how far back timestamp may be set (for example to within one interval of block.timestamp), or emit the effective floor alongside the anchor so monitoring can detect a neutralised guard. dropGuardStatus should expose the effective floor rather than only the raw snapshot.
Status	Acknowledged. Accepted as a governance-controlled setter following the same pattern as the oracles. The finding carries over to setNavGuardSnapshot in a changed and more serious form: the parameter now sets driftFrom as well, and _currentBand scales the whole band — floor included — by the accrued drift, so a backdated stamp makes the floor clamp honest readings **upward**. Measured: a block number pasted where a timestamp belongs reports **533** for a position worth **100**, and the first accrual writes that figure into storage. The response addresses the previous consequence (a silently disabled guard) rather than the current one (an actively over-reported NAV).

[I05] The Centrifuge family has no proxy instantiation path, so initialization is not atomic	
Contract	deploy/08_DeployCentrifuge.s.sol, deploy/06_DeployAssetVault.s.sol

Severity	Informational

Description	08_DeployCentrifuge.s.sol deploys three things - the adapter, the YieldGroupCentrifuge implementation, and the family beacon - and does not create a BeaconProxy.
	
	JavaScript
	`function deployCentrifuge(address governance) public returns (address adapter, address impl, address beacon) {`
	`  adapter = address(new AdapterCentrifuge());`
	`  impl = address(new YieldGroupCentrifuge());`
	`  UpgradeableBeacon b = new UpgradeableBeacon(impl);  // no BeaconProxy anywhere`
	`  ...`
	
	The Beacons struct in 06_DeployAssetVault.s.sol carries only hub / core / frv / flux, and the code-length guards at :120-123 cover only those four. Outside 08 itself there is no mention of Centrifuge anywhere in deploy/ or contracts/registry/. The other three families create and initialize their proxy atomically, so an uninitialized proxy is structurally unrepresentable:
	
	JavaScript
	`core = address(new BeaconProxy(beacons.core, abi.encodeCall(YieldGroup.initialize, (hub, p.asset, p.blocksPerYear, acm))));`
	`frv  = address(new BeaconProxy(beacons.frv, abi.encodeCall(YieldGroupFRV.initialize, (hub, p.asset, acm))));`
	`flux = address(new BeaconProxy(beacons.flux, abi.encodeCall(YieldGroup.initialize, (hub, p.asset, 0, acm))));`
	
	For Centrifuge that guarantee degrades from a code-enforced property to operational discipline.
	
	Impact
	There is no contract defect. initialize carries the initializer modifier, the implementation calls _disableInitializers() in the base constructor (YieldGroupBase.sol:73), all three addresses are zero-checked, IHub(hub_).asset() == asset_ is cross-verified, and __ReentrancyGuard_init() is called. Storage layout was verified empirically with forge inspect and all four gap segments align exactly.
	The exposure exists only if the proxy is created manually in two transactions. Front-running the second one causes the deployer's initialize to revert with InvalidInitialization - a loud failure. If operations proceed past that failure and still register the proxy with the Hub, the attacker controls both _hub (checked by onlyHub, YieldGroupBase.sol:66) and _accessControlManager (used by

	_checkAccessAllowed, :506), and can extract everything the Hub routes into that group via setInnerDepositQueue, addResource with a malicious adapter, or sweep. Note the HubAssetMismatch check offers no protection here: the attacker passes the real Hub and real asset and only substitutes the third argument. A poisoned proxy returns correct values from hub() and asset(); the only difference is accessControlManager(). If the deployment flow re-runs a script that silently skips already-initialized proxies, no failed transaction remains to raise the alarm.
Recommendation	Add a centrifuge field to the Beacons struct in 06_DeployAssetVault.s.sol (with the same code-length guard as :120-123) and create the proxy atomically as the other families do: JavaScript `centrifuge = address( new BeaconProxy(beacons.centrifuge, abi.encodeCall(YieldGroupCentrifuge.initialize, (hub, p.asset, acm))) );` Until then, record a hard operational requirement: the Centrifuge proxy must be created in a single transaction carrying the initialize calldata; a two-step create-then-initialize is prohibited. This finding sits inside the blind spot described in the audit limitations: the complete onboarding path for Centrifuge - proxy instantiation, Hub registration and ACM role grants - is absent from this PR and could not be audited.
Status	Fixed in commit 6841b08. The proxy is now created and initialized in one call, with code checks on both the beacon and the hub: JavaScript `require(beacon.code.length > 0, "DeployCentrifugeSource: beacon has no code"); require(hub.code.length > 0, "DeployCentrifugeSource: hub has no code"); source = address(new BeaconProxy(beacon, abi.encodeCall(YieldGroupCentrifuge.initialize, (hub, asset, acm))));`

[I06] The Spoke binding is the one price-guard field the setter does not validate	
Contract	YieldGroupCentrifuge.sol
Severity	Informational

Description	setPriceAgeGuard derives poolId, scId and assetId from the vault, but takes spoke as a free argument and stores it after checking only that it is non-zero. Freshness is then read from that address while the price comes from the vault, so nothing enforces that the two describe the same market.
	
	JavaScript
	`// YieldGroupCentrifuge.sol:207`
	`if (spoke == address(0)) revert InvalidPriceAgeGuard();`
	
	`// YieldGroupCentrifuge.sol:373-374 - freshness read from the stored address`
	`IAdapterCentrifuge(adapter).priceTimestamps(guard.spoke, guard.poolId, guard.scId, guard.assetId);`
	
	The interface reasons about exactly this failure mode and designs the other three fields around it (IYieldGroupCentrifuge.sol:298-303):
	
	JavaScript
	`"a contradicting one that happened to name a live market would age a marker that has nothing to do with this position, leaving the guard permanently satisfied. Only the Spoke stays configurable, because that address can genuinely move."`
	
	The protection was deliberate for the three fixed identifiers and absent for the one the same sentence calls mobile — the field most likely to drift out of date.
	
	Impact
	The price age guard can attest to the freshness of a market other than the one being valued, while priceAgeGuard() continues to report it enabled. No view exposes the mismatch between the configured Spoke and the one actually serving the vault.
	
	Why Informational
	A Spoke with no markers returns computedAt == 0 and reverts StalePrice immediately, which any fork simulation catches. A Spoke that does answer is frozen from that moment, so with a non-zero maxAge the binding trips StalePrice on its own within that window — the error announces itself, only late. Permanent silence needs maxAge == 0, which the NatSpec documents as "trust Centrifuge's own window and add nothing": a mode in which Venus has already declined the check the stalebinding would defeat.
	
	The setter also does not require spoke.code.length > 0.

Recommendation	Derive the Spoke from a canonical Centrifuge source, or cross-check the supplied address against one, so it is validated to the same standard as the three identifiers beside it. At minimum require spoke.code.length > 0, and add re-validation to the runbook for a Spoke redeployment, since the design already anticipates that address moving.

Status	Fixed in commit c8b595c. The unvalidated spoke parameter no longer exists — it was removed together with the price age guard.